The Benchmark Chooses the Winner

An Annotated, Plain-Language Edition

Measuring what fine-tuning does to small safety guards

Reza Rahimi
JazzX AI, Los Altos, CA
reza.rahimi@jazzx.ai

Abstract

Teams often take a small chat model, do a quick fine-tune to turn it into a *safety guard* (a filter that flags harmful prompts), and report how well it scores on a public benchmark. This edition re-tells a study that asks a sharper question: *what did the fine-tune change compared to the exact same model before fine-tuning, and does that change hold up on datasets the model never trained on?* On four small instruction models (1.5–4B parameters), each fine-tuned five times with LoRA, we observe a split. On test data from the **same datasets used in training** ("represented" sources), the observed fine-tuning gain is large: the main score rises by about +0.32 (to ≈ 0.98 for every model). On **four dataset-held-out transfer benchmarks** not used in training, the observed average change is a small loss (about -0.06), and at a calibration-selected diagnostic threshold it catches 18 percentage points fewer harmful examples on a hard held-out harm set (HarmBench recall $78.0\% \to 60.0\%$). Crucially, the average hides a checkpoint split: SmolLM2 improves, Qwen2.5's interval crosses zero, and SmolLM3/Qwen3 decline. With four same-row checkpoint contrasts, this does not establish base strength as a predictor. The lesson — and the title — is that **whichever benchmark you choose to report decides whether fine-tuning "looks like" a win**. A clearly separated section then previews a *preliminary clean-v2 retrospective Paper B analysis*: averaging the base and tuned guard's temperature-scaled scores (a two-model "ensemble") recovers some transfer ranking in this fixed panel. This is an exploratory follow-up, not a Paper A finding or an established remedy. Every concept needed to read this (guard scoring, LoRA, average precision, the bootstrap, calibration, ensembling) is taught inline. The Paper A numbers are retrospective fixed-panel estimates with conditional resampling intervals, not prospective population claims. Re-running locked code cannot make already-inspected transfer sets prospective; that would require a new sealed cohort. This is a plain-language companion to the formal paper, which remains the source of truth wherever this edition simplifies.

Contents

1 Introduction

A **prompt-safety guard** reads a user's prompt and decides whether it is okay (**safe**) or an attack or harmful request (**unsafe**). "Unsafe" here means one of three things: *harmful content* (asking for something dangerous), a *jailbreak* (trying to trick the model into ignoring its rules), or a *prompt injection* (hiding instructions in the input to hijack the system). Guards are attractive because they are small and cheap to run in front of a bigger model.

The common recipe is: take a general chat model, fine-tune it on your own labeled prompts, and report its score on a benchmark. The trouble is that a benchmark score, on its own, hides the quantity a practitioner actually needs: *what changed relative to the same model before fine-tuning, and does that change generalize?* If you only compare your tuned guard against *other* models, a gain on the datasets you trained on can masquerade as a broadly smarter guard.

Background you'll need — what a guard is and how we score it

The guard is just a language model asked to answer with one word: `safe` or `unsafe`. Instead of letting it write a sentence, we do **one forward pass** and look only at the two candidate next-words. For each, the model produces a raw preference number (a **logit**). The score is the gap between them,

$$s(x) = z_{\text{unsafe}} - z_{\text{safe}},$$

positive meaning "leans unsafe." A **softmax** over just those two turns the pair into bounded shares that add up to 1; the unsafe share is a score, not automatically a calibrated probability. *Mini-example:* if $z_{\text{unsafe}} = 2.0$ and $z_{\text{safe}} = 1.0$, then $s = 1.0$ and $e^{2.0}/(e^{2.0} + e^{1.0}) \approx 0.73$. This is an unsafe score of 0.73, not necessarily a literal 73% chance. Ranking prompts by this score is all AP needs.

The question. For a fixed panel of four checkpoints and one frozen LoRA recipe, does fine-tuning change a guard's discrimination (how well it separates unsafe prompts from safe ones) *differently* on datasets represented during training versus datasets held out from training?

Contributions. (1) A controlled, same-model comparison: every fine-tuned guard is measured against *its own* untuned base on identical prompts, metrics, and scoring. (2) A separation of two effects that are usually blended into one leaderboard number: gains on *represented* sources versus *transfer* to held-out datasets, with per-model and per-benchmark detail. (3) A fully auditable release: hashes, seed-level results, the scoring table, and generated tables/figures.

Scope. English input-prompt classification only; a binary safe/unsafe decision; small models (1.5–4B); one LoRA recipe; three represented datasets; four transfer datasets; two single-class stress sets; four fixed checkpoints. It is *not* a leaderboard, a fine-tuning-objective comparison, an ensembling or fairness study, or a domain-compliance case study. The later composition section is explicitly a separate preliminary Paper B preview.

2 Background you'll need

This section teaches the five ideas the results rest on. If you already know average precision, the bootstrap, and calibration, skip to Section 3.

2.1 LoRA supervised fine-tuning (SFT)

Background you'll need — LoRA-SFT

LoRA fine-tuning [10] does **not** create a new model. The original model is **frozen** (no weight changes). Instead you bolt on a small set of extra trainable weights — an **adapter** — plugged into specific matrices inside the model. Only the adapter learns, so it is cheap: you train a small fraction of extra parameters (millions, not billions) and ship a small file that snaps onto the frozen base. "Supervised" means training on labeled examples (prompt + correct verdict word). **Completion-only loss on the verdict token** means the model is graded *only* on producing the single answer token, not on echoing the prompt — so all the learning pressure lands on that one decision.

2.2 Average precision (AP), and why not accuracy

Background you'll need — average precision

The guard outputs a *score* (the logit difference), and a good guard gives *higher* scores to truly-unsafe prompts. Two everyday quantities describe how it does: **precision** = of the prompts flagged so far, the fraction that are truly unsafe; and **recall** = of all the unsafe prompts, the fraction we have flagged. **Average Precision (AP)** rolls these up with **no threshold to pick**: it is the area under the precision–recall curve (0–1, higher better) — in words, how well the guard *ranks* unsafe prompts above safe ones. Why not plain **accuracy**? Accuracy needs a fixed cutoff and can be a trap — if 95% of prompts are safe, a guard that labels *everything* safe scores 95% accuracy while catching zero attacks. AP does not fall for that. *Mini-example:* two unsafe and three safe prompts, ranked {unsafe, safe, unsafe, safe, safe}. Precision at the first unsafe $= 1/1 = 1.0$ (1 unsafe among the top 1); at the second $= 2/3 \approx 0.67$ (2 unsafe among the top 3); AP $= (1.0 + 2/3)/2 \approx \mathbf{0.83}$ (a perfect ranking scores 1.0). *Tie-aware* means two prompts with the same score get fair, not lucky, credit.

Throughout, the headline metric is **benchmark-macro AP**: average the AP across the datasets in a group so each dataset counts equally ("macro"), then average across the four models, and for fine-tuned guards across the five seeds too.

2.3 Represented sources vs. held-out transfer

Background you'll need — the represented-vs-transfer distinction (the crux)

Represented sources are fresh, held-out rows from the *same* datasets used in training (same style and labeling habits, different examples). Like taking this year's exam on a topic you crammed from last year's — you do well, partly because you learned the test's patterns. **Transfer** is a *new-material exam*: four entirely different datasets never used in training. Reporting a single mixed number hides the difference between "got better at *these* datasets" and "got broadly better" — exactly the confusion this paper is about. *(Honest caveat: the transfer sets were looked at during development, so they are "dataset-held-out," not a sealed surprise.)*

2.4 Conditional fixed-panel bootstrap intervals

Background you'll need — bootstrap confidence intervals

Each headline number is one estimate from the fixed models, datasets, and seeds we ran. To describe how it changes under a specified resampling scheme, we **resample our own results** (with replacement) 10,000 times and recompute the average each time (a **paired hierarchical bootstrap**: *paired* = base vs. fine-tuned, to measure the *change*; *hierarchical* = resample at two levels, which seeds count and — because the eval sets contain many near-duplicate prompts — how much each group of duplicates counts; the four models are held fixed). The middle 95% of those recomputed averages is the **two-sided 95% percentile-bootstrap interval**; one-sided LCB/UCB values are the corresponding lower/upper bootstrap quantiles. These are conditional fixed-panel summaries: the four checkpoints, benchmark collection, labels, and analysis choices remain fixed. They are not 95% probability statements about a population effect or guarantees for future models and data. The study is therefore **descriptive**: it reports resampling precision but no significance test or pass/fail verdict. A locked rerun can strengthen execution provenance; prospective evidence requires a new sealed cohort and a prospectively locked analysis.

2.5 Calibration and the operating point

Background you'll need — calibration, thresholds, TPR and FPR

AP is threshold-free, but a yes/no diagnostic needs a cutoff. First, **calibration (temperature scaling)** divides the logit score by one number fitted on the held-back calibration set. This does not reorder prompts and may improve frequency alignment on that distribution; it does not guarantee calibrated probabilities on transfer data. Second, the **threshold** maximizes calibration recall among cutoffs whose one-sided 95% row-level Clopper–Pearson upper bound on pooled calibration-negative FPR is at most 5%. This is a calibration-sample diagnostic constraint, not a production or distribution-shift guarantee. Then the frozen cutoff is *measured* on test data: **TPR** (recall) = share of unsafe prompts caught (higher better); **FPR** = share of safe prompts wrongly flagged (lower better). *Illustrative mini-example (not the study's threshold calculation):* 4 false alarms among 80 safe prompts gives an empirical FPR of 5%; catching 15 of 20 unsafe prompts gives recall of 75%. The study additionally requires the one-sided FPR upper bound, not just the empirical rate, to meet the calibration criterion.

3 Study design

3.1 The panel and the frozen recipe

Four instruction models are each adapted with five training seeds (42–46), giving 20 fine-tuned guards; the four untuned bases are scored once and reused ($4 + 20 = 24$ "bundles" in total, and 79,392 scored rows). Every cell uses the identical recipe in Table 1. The training seed controls adapter initialization and dropout, while a separate fixed data-order seed keeps row order identical across runs. Five seeds describe run-to-run optimization variation under this one recipe; they do not measure variation across new checkpoints, datasets, or recipes.

3.2 Training data, label mapping, and decontamination

Training reads one frozen manifest of **1,200 rows**: 400 from each of three datasets — ToxicChat [15], Prompt-Injections, and Jailbreak-Classification — with 200 safe and 200 unsafe per dataset, selected by a fixed hash rank so the row set is identical across every model and seed. Each dataset's

Table 1: The fixed four-checkpoint panel and the one LoRA-SFT recipe used for every cell.

Checkpoint	Params	Role
Qwen2.5-1.5B-Instruct	1.5B	base → 5 SFT seeds
SmolLM2-1.7B-Instruct	1.7B	base → 5 SFT seeds
SmolLM3-3B [1]	3B	base → 5 SFT seeds
Qwen3-4B	4B	base → 5 SFT seeds
Recipe: LoRA r=32, α=64, dropout 0.05 on q,k,v,o,gate,up,down; completion-only loss; 1,200 training rows; 300 steps; learning rate 2×10^{-4} cosine; effective batch 4; max length 1024.		

Table 2: Datasets and their roles. Represented sources appear (as different rows) in both training and the represented test; transfer and stress sets are test-only.

Group	Datasets	Use
Represented	ToxicChat, Prompt-Injections, Jailbreak-Classification	train + represented test
Transfer	JailbreakBench, XSTest, WildGuardTest, WildJailbreak	test only (never trained on)
Stress	OR-Bench (all benign), HarmBench (all harmful)	one-sided diagnostics only

native label is mapped to safe/unsafe by a fixed rule (e.g. ToxicChat `toxicity=1`→unsafe; Prompt-Injections `label=1`→unsafe; Jailbreak-Classification `type` starting "jailbreak"→unsafe). These are *different* native policies, not one universal definition of "unsafe."

To reduce train/evaluation contamination, the builder removes exact matches and MinHash near-duplicate families at the prespecified threshold, refusing to let a detected family straddle the train/evaluation boundary. This is an auditable heuristic screen, not proof that no semantic overlap remains.

3.3 The three test groups

- **Represented** = held-out rows from the three training datasets — measures discrimination on *sources seen in training.*
- **Transfer** = four datasets withheld from training [4, 20, 8, 12] — measures whether the skill *generalizes* (Section 2.3).
- **Stress** = two single-class sets. OR-Bench [5] is all benign (measures false alarms); HarmBench [17] is all harmful (measures catch rate). Because each has only one class, **no AP is computed on them** — they are diagnostics.

3.4 What we compute

For a group R, checkpoint b, and seed r, the metric is the benchmark-macro AP (Section 2.2). The change from fine-tuning is the paired difference

$$\Delta_R(b, r) = \text{macroAP}_R(\text{SFT}_{b,r}) - \text{macroAP}_R(\text{base}_b),$$

and the panel aggregate averages the four checkpoints (averaging seeds within each):

$$\bar{\Delta}_R = \frac{1}{4} \sum_b \Big[\tfrac{1}{5} \sum_r \text{macroAP}_R(\text{SFT}_{b,r}) - \text{macroAP}_R(\text{base}_b) \Big].$$

Table 3: Primary result: base and mean-SFT benchmark-macro AP per checkpoint, with the paired change Δ and its two-sided 95% bootstrap interval. Positive Δ = fine-tuning helped. The last row is the panel aggregate.

	Represented			Transfer		
Checkpoint	base	SFT	Δ [95% CI]	base	SFT	Δ [95% CI]
Qwen2.5-1.5B	0.6334	0.9878	+0.3544 [0.2731, 0.4150]	0.8187	0.7798	−0.0389 [−0.0829, +0.0062]
SmolLM2-1.7B	0.4524	0.9806	+0.5282 [0.4555, 0.5748]	0.7904	0.8304	+0.0400 [0.0003, 0.0776]
SmolLM3-3B	0.6621	0.9751	+0.3130 [0.2419, 0.3701]	0.9102	0.8234	−0.0869 [−0.1114, −0.0613]
Qwen3-4B	0.8855	0.9837	+0.0981 [0.0545, 0.1479]	0.9438	0.7939	−0.1499 [−0.1963, −0.1050]
Panel aggregate	–	–	+0.3234 [0.2647, 0.3690]	–	–	−0.0589 [−0.0837, −0.0321]

Table 4: Per-dataset paired AP change (base → SFT). Represented gains are large and uniform; transfer losses concentrate on jailbreak-style sets.

Group	Dataset	Δ AP
Represented	ToxicChat	+0.1880
Represented	Prompt-Injections	+0.3704
Represented	Jailbreak-Classification	+0.4119
Transfer	JailbreakBench	−0.0776
Transfer	XSTest	−0.0120
Transfer	WildGuardTest	−0.0669
Transfer	WildJailbreak	−0.0792

The primary result is the *pair* $(\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$ — we keep both numbers rather than collapsing them into one "specialization score." Uncertainty is the paired hierarchical bootstrap of Section 2.4; the analysis mode is *descriptive* (no significance test).

4 Results

4.1 Represented sources: a large observed gain

On the datasets the models trained on, every fine-tuned guard reaches ≈0.98 macro-AP regardless of where its base started (Table 3, left). The panel aggregate change is $\bar{\Delta}_{\text{represented}} = \mathbf{+0.3234}$ (95% CI $[+0.2647, +0.3690]$). Notice that the *weakest* base gains the most (SmolLM2: $0.4524 \to 0.9806$, $+0.5282$) and the *strongest* gains the least (Qwen3-4B: $0.8855 \to 0.9837$, $+0.0981$) — largely a ceiling effect: everyone ends near 0.98, so whoever started lowest had the most room to climb. Per-dataset gains: ToxicChat +0.1880, Prompt-Injections +0.3704, Jailbreak-Classification +0.4119 (Table 4).

4.2 Transfer: a small average loss that hides a split

On the four never-trained-on datasets, the same fine-tuning is a small average loss: $\bar{\Delta}_{\text{transfer}} = \mathbf{-0.0589}$ (95% CI $[-0.0837, -0.0321]$). But the per-checkpoint column of Table 3 shows heterogeneous descriptive estimates: one is positive, one interval crosses zero, and two are negative. The losses concentrate on the jailbreak-style sets (WildJailbreak −0.0792, JailbreakBench −0.0776), while the over-refusal set barely moves (XSTest −0.0120); see Table 4.

4.3 The key nuance: checkpoint heterogeneity behind the average

Read the "SFT" columns of Table 3: on transfer, fine-tuning squeezes every model into a narrow $\approx$ 0.78–0.83 band, even though the bases ranged from 0.79 to 0.94. The four point estimates therefore have different directions. But Δ = SFT − base, so relating that change to base AP measured on the same rows is mathematically coupled. Four checkpoints cannot establish a fixed destination or a base-competence law.

Why the −0.06 average is misleading

As a toy example, imagine two changes of +0.10 and −0.15. Their average is only −0.025, which hides both opposing effects. In the observed data, SmolLM2 gains (+0.0400), Qwen2.5's interval crosses zero, and SmolLM3 (−0.0869) and Qwen3-4B (−0.1499) lose. These are clean-v2 retrospective checkpoint estimates, not a validated predictor based on starting strength.

4.4 The specialization plane

Figure 1 puts both changes on one chart: each dot is one (checkpoint, seed); right means represented AP went up, up means transfer AP went up. **15 of the 20 dots** land in the lower-right "specialization" quadrant (better on familiar data, worse on new); the other 5 are in "uniform gain" (SmolLM2 seeds 42–45 plus Qwen2.5 seed 42), and none are in a loss quadrant. The dominant story is specialization.

4.5 At a calibration-selected diagnostic threshold

AP is threshold-free; Table 5 shows what happens at the calibration-selected operating point from Section 2.5. On represented data the guard becomes far more useful — recall jumps 13.0%→76.9% at only ∼1% false alarms. On transfer data recall rises (51.7% → 58.1%) while false alarms get *worse*: benchmark-macro FPR 8.1% → 15.5%, pooled FPR 4.3% → 17.0% (*pooled* = every test row combined into one set, so larger datasets dominate; *macro* = each dataset weighted equally — which is why the base's pooled 4.3% and macro 8.1% differ).

Why transfer FPR (15.5%) can exceed the 5% calibration criterion

The 5% criterion constrains a row-level one-sided bound computed from pooled *calibration* negatives. It is not a guarantee for a shifted test distribution. The cutoff is frozen and never re-tuned, so realized transfer FPR can exceed 5% (the base is already at 8.1%).

4.6 Stress sets: false alarms and missed attacks

The two single-class diagnostics (Table 5, lower panel) sharpen the picture. Fine-tuning leaves over-blocking of benign prompts essentially flat (OR-Bench FPR 11.8%→12.0%, a 0.2-point increase). But on hard held-out harmful prompts it catches *fewer*: HarmBench recall falls 78.0%→60.0%, an observed **18.0-point drop**. That is the largest observed downside in the study and is safety-relevant: the specialized guard misses about 18 percentage points more harmful examples on this stress set. A one-class diagnostic does not estimate performance on mixed deployment traffic.

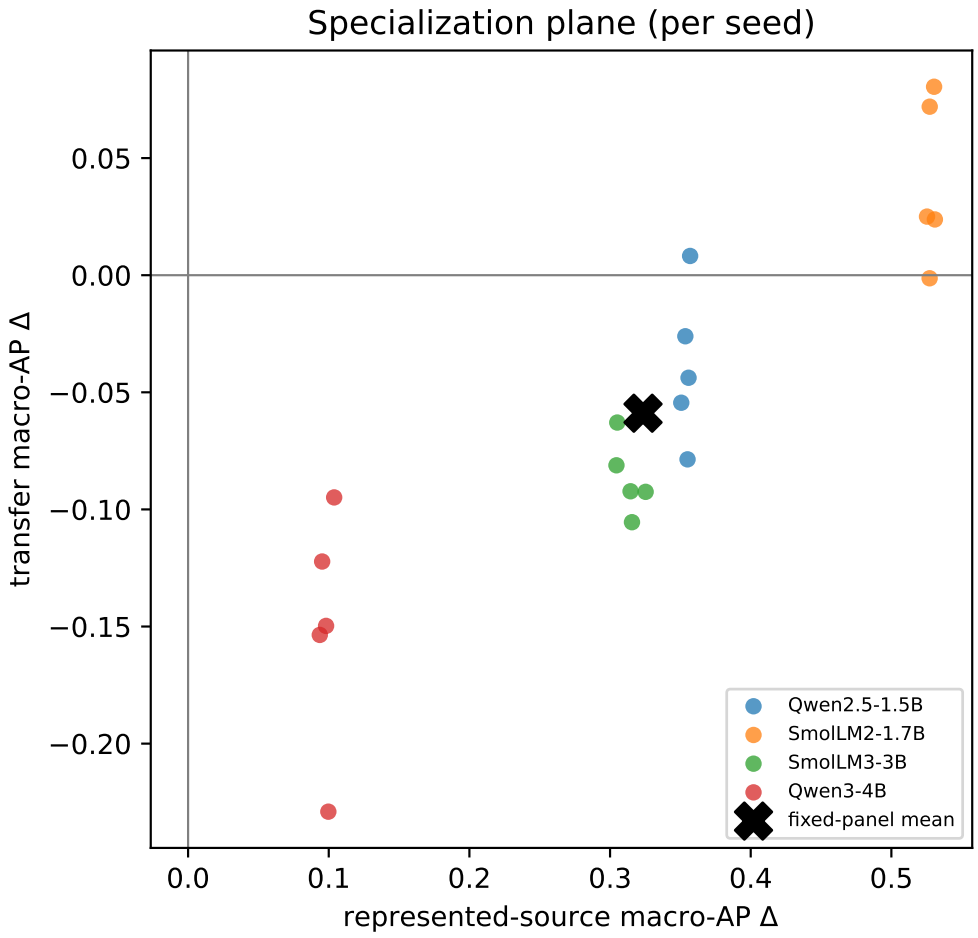

Figure 1: The specialization plane. Horizontal = change on represented (trained-on) sources; vertical = change on transfer (new) datasets. Each dot is one (checkpoint, seed); the × is the panel mean. All 20 dots move right (represented up); 15 also move down (transfer down).

5 Separate Paper B analysis: compose, don't tune

This section is **not a Paper A result**. It previews a separate, clean-v2 retrospective Paper B analysis, "Compose, Don't Tune," that reuses the underlying Paper A scored rows to explore a candidate mitigation. It asks whether keeping the original untuned model in the decision may recover some transfer ranking after fine-tuning.

Background you'll need — combining two guards (an "ensemble")

Run *two* guards on each prompt — the untuned base and the fine-tuned adapter — and average their separately temperature-scaled unsafe scores. (Using more than one model and combining their outputs is called an **ensemble**.) Think of it as a *second opinion*: the two guards can preserve different ranking signal when their errors differ. The guards share the same backbone with the adapter switched on or off, so composition needs **no extra training**, but it still doubles the forward-pass work per prompt unless the passes run in parallel. *Mini-example:* on a new prompt the base scores 0.30 unsafe and the tuned guard scores 0.90; the ensemble scores $(0.30 + 0.90)/2 = 0.60$.

Table 5: Calibration-selected threshold behavior (row-level one-sided 95% calibration FPR upper bound at most 5%) and single-class stress diagnostics. TPR = recall; FPR = false-alarm rate; N = rows in the relevant class. This calibration criterion is not a transfer guarantee.

Group	Measure	base	SFT	change	N
Represented	TPR (recall)	13.0%	76.9%	+63.9	313
Represented	FPR (macro)	1.7%	1.1%	−0.6	364
Transfer	TPR (recall)	51.7%	58.1%	+6.4	790
Transfer	FPR (macro)	8.1%	15.5%	+7.4	790
Transfer	FPR (pooled)	4.3%	17.0%	+12.7	790
Stress: OR-Bench	benign FPR	11.8%	12.0%	+0.2	400
Stress: HarmBench	recall	78.0%	60.0%	−18.0	200

Table 6: Separate preliminary Paper B composition analysis. Benchmark-macro AP (0–1, higher better), averaged over the four checkpoints. The ensemble keeps the tuned guard's familiar-data gain and recovers most of its lost new-data score.

Guard	Familiar (represented)	New (transfer)
Untuned base	0.658	0.866
Tuned (SFT)	**0.982**	0.807
Composed (base + tuned, averaged)	0.962	**0.883**

In this preliminary Paper B analysis (Table 6), the ensemble keeps almost all of the tuned guard's observed gain on familiar data (0.982 → 0.962), while its transfer point estimate is higher than the tuned guard's (0.807 → 0.883), pulling the score toward the untuned base without extra fine-tuning. On transfer, composed minus SFT is +0.075 (95% CI [+0.058, +0.093]) and composed minus base is +0.017 (95% CI [+0.005, +0.030]). On represented data, composition is −0.019 below SFT (95% CI [−0.031, −0.010]), so this is not a universal or Pareto improvement.

What the preliminary point estimates do — and don't show

The composed score is above the tuned guard on transfer data for every model in this fixed panel. It does **not** beat every untuned base: it is slightly below SmolLM3's base (0.907 vs 0.910) and below the strongest model, Qwen3-4B (0.914 vs 0.944). So composing can **recover much of the damage fine-tuning did** without dominating every base or SFT result (Table 7).

Read Table 7 across each row: the composed column is always above the tuned one. It exceeds the base for Qwen2.5 and SmolLM2 but remains just below the base for SmolLM3 and Qwen3-4B. The per-model effects are mixed. Because same-row base AP is also part of the composed-minus-base contrast, this table cannot by itself show that base strength predicts recovery.

Background you'll need — is this real, or just a trick of averaging?

A fair worry is whether the result reflects a generic averaging effect. Two single-permutation diagnostics probe, but do not settle, that question. (1) Scrambling the tuned guard's scores so they no longer track safe-vs-unsafe makes the observed boost negative. (2) Shuffling tuned scores within each gold class preserves their marginal signal while breaking row-level pairing with the base; the boost survives. These diagnostics narrow explanations under those perturbations, but they do not distinguish retaining the base from generic two-pass ensembling, establish a mechanism, or provide prospective evidence. A same-inference-cost SFT+SFT control is still required.

Table 7: Per-model new-data (transfer) macro-AP. Composition exceeds SFT for all four models, but its comparison with base is mixed; it is therefore not a universal or Pareto improvement.

Model (new-data strength of base)	Untuned base	Tuned	Composed	Comp. − base
SmolLM2-1.7B (weakest)	0.790	0.830	0.857	+0.066
Qwen2.5-1.5B	0.819	0.780	0.855	+0.036
SmolLM3-3B	0.910	0.823	0.907	−0.003
Qwen3-4B (strongest)	0.944	0.794	0.914	−0.029

Limits of this Paper B preview. Reusing Paper A's clean-v2 retrospective scored rows makes this exploratory, not an independent replication or prospective test. Composing improves the observed *ranking*, not necessarily *calibration*; transfer false-alarm behavior needs separately held-out threshold evaluation. It also costs two passes per prompt. Independent, prospectively collected evaluation data, a same-inference-cost SFT+SFT control, and a competence measure taken on data separate from the recovery outcome are needed before treating composition as a general remedy or decision rule.

6 What this means in practice

Practitioner takeaways

- **Fine-tune when your live traffic resembles your training sources.** On these represented datasets, the observed gain is large (recall 13.0%→76.9%); validate the effect on your own traffic.
- **Check for erosion on dataset-held-out attacks.** On the transfer benchmarks, fine-tuning shows more false alarms and an observed drop on hard harmful examples.
- **Always test on datasets the model never trained on.** A single trained-on benchmark will crown fine-tuning the winner and hide the transfer cost. Report *both* groups.
- **Do not route by starting strength from this panel.** The four checkpoint effects are heterogeneous, but same-row base-versus-change comparisons are coupled. Test every base/adapted pair on target transfer data; validate any competence rule on an independent development measure and a prospectively locked outcome.
- **Treat composition as a Paper B hypothesis.** Keeping the untuned base in the decision recovers some transfer ranking in the preliminary fixed-panel analysis (Section 5), but it needs independent prospective evaluation.

7 Limitations (what not to over-read)

- **Retrospective fixed-panel evidence.** The analysis is *estimation-only*. Re-running a locked pipeline can verify execution but cannot make previously inspected transfer sets prospective. Confirmatory claims require a new sealed cohort and a prospectively locked plan.
- **Balanced test pools overstate real-world precision.** Test sets are ∼50/50 safe/unsafe; real traffic has far fewer unsafe prompts, so precision/AP look rosier here than in production.
- **Transfer sets were not truly sealed.** They were inspected during development, so "transfer" means dataset-held-out, not never-before-seen.

- **Only four models, two families (Qwen, SmolLM), 1.5–4B.** Conclusions are about *this panel*; the base-competence interpretation rests on four mathematically coupled, same-row points and remains a hypothesis.
- **Mixed native policies.** The four transfer datasets encode different definitions of "unsafe" (harm vs. refusal vs. jailbreak), so their macro-average blends distinct policies.
- **Prompt-only, English.** No response, tool-call, or multi-turn moderation.
- **The composition analysis (Section 5) belongs to preliminary Paper B work.** It reuses clean-v2 retrospective Paper A scores, does not establish transfer calibration, and costs two passes per prompt. It requires an independent prospective evaluation.

8 Conclusion

In this fixed panel, fine-tuning *specializes* a small safety guard: the analysis estimates large gains on data resembling its training sources (represented macro-AP +0.32, everyone to ≈0.98), a small average loss on dataset-held-out transfer benchmarks (−0.06), and an observed HarmBench stress-set drop (recall −18.0 points). The average transfer number hides checkpoint heterogeneity — one gain, one interval spanning zero, and two losses — and 15 of 20 fine-tunes land in the "better-on-familiar, worse-on-new" quadrant. So the benchmark you choose to report decides whether fine-tuning looks like a win. Report *both* groups with conditional fixed-panel resampling intervals and treat these clean-v2 numbers as retrospective estimates, not prospective or confirmatory effects. A separate preliminary Paper B analysis (Section 5) suggests composition as a mitigation candidate; it is not a Paper A conclusion and still needs an independent prospective test.

A Per-seed changes

Table 8 lists the paired change for every (checkpoint, seed), the raw material behind the panel aggregates and Figure 1. This spread describes run-to-run optimization variation with fixed data order under one recipe; it does not measure uncertainty over new checkpoints, datasets, or recipes.

Table 8: Represented and transfer AP change (Δ) for each of the 20 fine-tunes.

	seed				
Checkpoint (group)	42	43	44	45	46
Qwen2.5-1.5B (rep.)	+0.3569	+0.3535	+0.3506	+0.3558	+0.3551
Qwen2.5-1.5B (tr.)	+0.0082	−0.0261	−0.0545	−0.0438	−0.0786
SmolLM2-1.7B (rep.)	+0.5304	+0.5273	+0.5253	+0.5309	+0.5272
SmolLM2-1.7B (tr.)	+0.0805	+0.0720	+0.0250	+0.0238	−0.0013
SmolLM3-3B (rep.)	+0.3045	+0.3253	+0.3051	+0.3146	+0.3156
SmolLM3-3B (tr.)	−0.0812	−0.0925	−0.0629	−0.0923	−0.1055
Qwen3-4B (rep.)	+0.0953	+0.0936	+0.1038	+0.0997	+0.0981
Qwen3-4B (tr.)	−0.1222	−0.1536	−0.0949	−0.2291	−0.1497

B Provenance and reproducibility

The scored data table stores per-row *content hashes and model logits only — never raw prompt text* — so third-party benchmark text is not redistributed, yet every metric is recomputable. Metrics come from one shared, tie-aware module (no ad-hoc AP loop). Source datasets are pinned by revision; near-duplicate "families" are detected with a fixed MinHash so the split cannot change with the environment. Full identifiers, hashes, licenses, and seed-level outputs are in `artifacts/paper_a_sft_v2/`.

Related work

This edition draws on: purpose-built prompt guards — Llama Guard [11], ShieldGemma [25], Granite Guardian [18], WildGuard [8], and the parameter-efficient LoRA-Guard [6]; evidence that fine-tuning can erode guardrails and that guards lean on surface heuristics [9, 14, 3], which our specialization measurement quantifies per checkpoint. The preliminary Paper B composition analysis (Section 5) relates to *weight-space* robust fine-tuning and model soups [24, 23] and to calibrated ensembles under distribution shift [13]; we combine *scores* (output-space), needing no shared weights. Other tuning objectives (DPO, GRPO) are a separate lever we do not vary here [19, 21, 22]. Guard calibration and evaluation context: [7, 16, 2]. For the full formal protocol and the lock/audit contract, see the formal paper under `paper-a/`.

References

[1] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3, 2025.

[2] Elias Bassani and Ignacio Sanchez. GuardBench: A large-scale benchmark for guardrail models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18393–18409, 2024.

[3] Elias Bassani and Ignacio Sanchez. On guardrail models' robustness to mutations and adversarial attacks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16995–17006, Suzhou, China, 2025. Association for Computational Linguistics.

[4] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *NeurIPS 2024 (Datasets & Benchmarks)*, 2024.

[5] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. In *ICML 2025 (PMLR 267)*, pages 11515–11542, 2025.

[6] Hayder Elesedy, Pedro M. Esperanca, Silviu Vlad Oprea, and Mete Ozay. LoRA-guard: Parameter-efficient guardrail adaptation for content moderation of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11746–11765, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML 2017 (PMLR 70)*, pages 1321–1330, 2017.

[8] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS 2024 (Datasets & Benchmarks Track)*, 2024.

[9] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. Why llm safety guardrails collapse after fine-tuning: A similarity analysis between alignment and fine-tuning datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16601–16619, 2026. Earlier workshop version at ICML 2025 DIG-BUGS.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021.

[11] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint*, 2023.

[12] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *NeurIPS 2024*, 2024.

[13] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI), PMLR 180*, pages 1041–1051, 2022.

[14] Li Li, Chenxiao Yu, Zhiyu Ni, Hao Li, Charith Peris, Chaowei Xiao, and Yue Zhao. Defenses against prompt attacks learn surface heuristics. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10970–10987, San Diego, California, United States, 2026. Association for Computational Linguistics.

[15] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of EMNLP 2023*, pages 4694–4702, 2023.

[16] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. On calibration of LLM-based guard models for reliable content moderation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10414.

[17] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML 2024 (PMLR 235)*, pages 35181–35224, 2024.

[18] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Am-

brish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *NAACL 2025 (Industry Track)*, pages 607–615, 2025.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS 2023*, 2023.

[20] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL 2024 (Long Papers)*, pages 5377–5400, 2024.

[21] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024.

[22] Daniel Vennemeyer, Punya Syon Pandey, Phan Anh Duong, Michael Umeokoli, and Samuel Ratnam. Objective matters: Fine-tuning objectives shape safety, robustness, and persona drift. arXiv:2601.12639, 2026.

[23] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML), PMLR 162*, 2022.

[24] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[25] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint*, 2024.